

```
import pandas as pd

df = pd.read_csv('PS/train.csv')

# Create an EDA summary table
eda_summary = pd.DataFrame({
    'Column': df.columns,
    'Total_Rows': len(df),
    'Non_Null_Count': df.notnull().sum().values,
    'Missing_Values': df.isnull().sum().values,
    'Data_Type': df.dtypes.values
})

print(eda_summary.to_string(index=False))
```

on_Null_Count	Missing_Values	Data_Type	Column	Total_Rows	N
40774	0	int64	Unnamed: 0	40774	
40774	0	int64	FarmerID	40774	
40774	0	object	State	40774	
40774	0	object	REGION	40774	
40774	0	object	SEX	40774	
40774	0	object	CITY	40774	
40774	0	int64	Zipcode	40774	
40774	0	object	DISTRICT	40774	
40774	0	object	VILLAGE	40774	
40774	0	object	MARITAL_STATUS	40774	
26255	14519	object	Location	40774	
26255	14519	object	Address type	40774	
26255	14519	object	Ownership	40774	
40774	0	int64	No_of_Active_Loan_In_Bureau	40774	
23101	17673	float64	Avg_Disbursement_Amount_Bureau	40774	
40774	0	float64	Non_Agriculture_Income	40774	
40713	61	float64	Total_Land_For_Agriculture	40774	
40774	K022-Village	category based on Agri parameters (Good, Average, Poor)		40774	
40774	0	object	K022-Nearest Mandi Name	40774	
40774	0	object	K022-Proximity to nearest mandi (Km)	40774	
40774	0	float64	K022-Proximity to nearest railway (Km)	40774	
40774	0	float64	K022-Village score based on socio-economic parameters (0 to 100)	40774	
40774	0	float64	K022-Village category based on socio-economic parameters (Good, Average, Poor)	40774	
40774	0	object	K022-Seasonal Average Rainfall (mm)	40774	
40774	0	float64	K022-Ambient temperature (min & max)	40774	
40774	0	object	R022-Village category based on Agri parameters (Good, Average, Poor)	40774	
40774	0	object	R022-Seasonal Average Rainfall (mm)	40774	
40774	0	float64			

			R022-Ambient temperature (min & max)	40774
40774	0	object		
			K021-Seasonal Average Rainfall (mm)	40774
40774	0	float64		
			K021-Ambient temperature (min & max)	40774
40774	0	object		
			R021-Seasonal Average Rainfall (mm)	40774
40774	0	float64		
			R021-Ambient temperature (min & max)	40774
40774	0	object		
			R020-Seasonal Average Rainfall (mm)	40774
40774	0	float64		
			R020-Ambient temperature (min & max)	40774
40774	0	object		
			Perc_of_house_with_6plus_room	40774
40617	157	float64		
			Women_15_19_Mothers_or_Pregnant_at_time_of_survey	40774
40617	157	float64		
			perc_of_pop_living_in_hh_electricity	40774
40617	157	float64		
			perc_Households_with_Pucca_House_That_Has_More_Than_3_Rooms	40774
40617	157	float64		
			mat_roof_Metal_GI_Asbestos_sheets	40774
40617	157	float64		
			perc_of_Wall_material_with_Burnt_brick	40774
40617	157	float64		
			Households_with_improved_Sanitation_Facility	40774
40617	157	float64		
			perc_Households_do_not_have_KCC_With_The_Credit_Limit_Of_50k	40774
40617	157	float64		
			K022-Total Geographical Area (in Hectares)-	40774
40774	0	float64		
			K022-Net Agri area (in Ha)-	40774
40774	0	float64		
			K022-Net Agri area (% of total geog area)-	40774
40774	0	float64		
			Kharif Seasons Irrigated area in 2022	40774
40774	0	float64		
			Kharif Seasons Cropping density in 2022	40774
40774	0	float64		
			Kharif Seasons Agricultural performance in 2022	40774
40774	0	float64		
			Kharif Seasons Agricultural Score in 2022	40774
40774	0	float64		
			Kharif Seasons Type of soil in 2022	40774
40774	0	object		
			Kharif Seasons Type of water bodies in hectares 2022	40774
40774	0	object		
			Kharif Seasons Agro Ecological Sub Zone in 2022	40774
40774	0	object		
			Kharif Seasons Seasonal average groundwater thickness (cm) in 2022	40774
40774	0	float64		
			Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2022	40774
40774	0	float64		
			Rabi Seasons Season Irrigated area in 2022	40774
40774	0	float64		

			Rabi Seasons Cropping density in 2022	40774
40774	0	float64		
			Rabi Seasons Agricultural performance in 2022	40774
40774	0	float64		
			Rabi Seasons Agricultural Score in 2022	40774
40774	0	float64		
			Rabi Seasons Type of soil in 2022	40774
40774	0	object		
			Rabi Seasons Type of water bodies in hectares 2022	40774
40774	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2022	40774
40774	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2022	40774
40774	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2022	40774
40774	0	float64		
			Rabi Seasons Kharif Season Irrigated area in 2021	40774
40774	0	float64		
			Rabi Seasons Cropping density in 2021	40774
40774	0	float64		
			Rabi Seasons Agricultural performance in 2021	40774
40774	0	float64		
			Rabi Seasons Agricultural Score in 2021	40774
40774	0	float64		
			Rabi Seasons Type of soil in 2021	40774
40774	0	object		
			Rabi Seasons Type of water bodies in hectares 2021	40774
40774	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2021	40774
40774	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2021	40774
40774	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2021	40774
40774	0	float64		
			Kharif Seasons Kharif Season Irrigated area in 2021	40774
40774	0	float64		
			Kharif Seasons Cropping density in 2021	40774
40774	0	float64		
			Kharif Seasons Agricultural performance in 2021	40774
40774	0	float64		
			Kharif Seasons Agricultural Score in 2021	40774
40774	0	float64		
			Kharif Seasons Type of soil in 2021	40774
40774	0	object		
			Kharif Seasons Type of water bodies in hectares 2021	40774
40774	0	object		
			Kharif Seasons Agro Ecological Sub Zone in 2021	40774
40774	0	object		
			Kharif Seasons Seasonal average groundwater thickness (cm) in 2021	40774
40774	0	float64		
			Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2021	40774
40774	0	float64		
			Kharif Seasons Kharif Season Irrigated area in 2020	40774
40774	0	float64		
			Kharif Seasons Cropping density in 2020	40774
40774	0	float64		

			Kharif Seasons Agricultural performance in 2020	40774
40774	0	float64		
			Kharif Seasons Agricultural Score in 2020	40774
40774	0	float64		
			Kharif Seasons Type of soil in 2020	40774
40774	0	object		
			Kharif Seasons Type of water bodies in hectares 2020	40774
40774	0	object		
			Kharif Seasons Agro Ecological Sub Zone in 2020	40774
40774	0	object		
			Kharif Seasons Seasonal average groundwater thickness (cm) in 2020	40774
40774	0	float64		
			Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2020	40774
40774	0	float64		
			Rabi Seasons Kharif Season Irrigated area in 2020	40774
40774	0	float64		
			Rabi Seasons Cropping density in 2020	40774
40774	0	float64		
			Rabi Seasons Agricultural performance in 2020	40774
40774	0	float64		
			Rabi Seasons Agricultural Score in 2020	40774
40774	0	float64		
			Rabi Seasons Type of soil in 2020	40774
40774	0	object		
			Rabi Seasons Type of water bodies in hectares 2020	40774
40774	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2020	40774
40774	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2020	40774
40774	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2020	40774
40774	0	float64		
			Night light index	40774
40774	0	float64		
			Village score based on socio-economic parameters (Non normalised)	40774
40774	0	float64		
			Village score based on socio-economic parameters (0 to 100)	40774
40774	0	float64		
			Village category based on socio-economic parameters (Good, Average, Poor)	40774
40774	0	object		
			Land Holding Index source (Total Agri Area/ no of people)	40774
40774	0	float64		
			Road density (Km/ SqKm)	40774
40774	0	float64		
			Target_Variable/Total Income	40774
40774	0	int64		

```
import pandas as pd
```

```
df = pd.read_csv('PS/test.csv')
```

```
# Create an EDA summary table
```

```
eda_summary = pd.DataFrame({
    'Column': df.columns,
    'Total_Rows': len(df),
```

```
    'Non_Null_Count': df.notnull().sum().values,  
    'Missing_Values': df.isnull().sum().values,  
    'Data_Type': df.dtypes.values  
})  
  
print(eda_summary.to_string(index=False))
```

on_Null_Count	Missing_Values	Data_Type	Column	Total_Rows	N
7196	0	int64	FarmerID	7196	
7196	0	object	State	7196	
7196	0	object	REGION	7196	
7196	0	object	SEX	7196	
7196	0	object	CITY	7196	
7196	0	int64	Zipcode	7196	
7196	0	object	DISTRICT	7196	
7196	0	object	VILLAGE	7196	
7196	0	object	MARITAL_STATUS	7196	
4685	2511	object	Location	7196	
4685	2511	object	Address type	7196	
4685	2511	object	Ownership	7196	
7196	0	int64	No_of_Active_Loan_In_Bureau	7196	
4079	3117	float64	Avg_Disbursement_Amount_Bureau	7196	
7196	0	float64	Non_Agriculture_Income	7196	
7186	10	float64	Total_Land_For_Agriculture	7196	
7196	K022-Village	category based on Agri parameters (Good, Average, Poor)		7196	
7196	0	object	K022-Nearest Mandi Name	7196	
7196	0	object	K022-Proximity to nearest mandi (Km)	7196	
7196	0	float64	K022-Proximity to nearest railway (Km)	7196	
7196	0	float64	K022-Village score based on socio-economic parameters (0 to 100)	7196	
7196	0	float64	K022-Village category based on socio-economic parameters (Good, Average, Poor)	7196	
7196	0	object	K022-Seasonal Average Rainfall (mm)	7196	
7196	0	float64	K022-Ambient temperature (min & max)	7196	
7196	0	object	R022-Village category based on Agri parameters (Good, Average, Poor)	7196	
7196	0	object	R022-Seasonal Average Rainfall (mm)	7196	
7196	0	float64	R022-Ambient temperature (min & max)	7196	
7196	0	object			

7196	0	float64	K021-Seasonal Average Rainfall (mm)	7196
7196	0	object	K021-Ambient temperature (min & max)	7196
7196	0	float64	R021-Seasonal Average Rainfall (mm)	7196
7196	0	object	R021-Ambient temperature (min & max)	7196
7196	0	float64	R020-Seasonal Average Rainfall (mm)	7196
7196	0	object	R020-Ambient temperature (min & max)	7196
7185	11	float64	Perc_of_house_with_6plus_room	7196
7185	11	float64	Women_15_19_Mothers_or_Pregnant_at_time_of_survey	7196
7185	11	float64	perc_of_pop_living_in_hh_electricity	7196
7185	11	float64	perc_Households_with_Pucca_House_That_Has_More_Than_3_Rooms	7196
7185	11	float64	mat_roof_Metal_GI_Asbestos_sheets	7196
7185	11	float64	perc_of_Wall_material_with_Burnt_brick	7196
7185	11	float64	Households_with_improved_Sanitation_Facility	7196
7185	11	float64	perc_Households_do_not_have_KCC_With_The_Credit_Limit_Of_50k	7196
7196	0	float64	K022-Total Geographical Area (in Hectares)-	7196
7196	0	float64	K022-Net Agri area (in Ha)-	7196
7196	0	float64	K022-Net Agri area (% of total geog area)-	7196
7196	0	float64	Kharif Seasons Irrigated area in 2022	7196
7196	0	float64	Kharif Seasons Cropping density in 2022	7196
7196	0	float64	Kharif Seasons Agricultural performance in 2022	7196
7196	0	float64	Kharif Seasons Agricultural Score in 2022	7196
7196	0	object	Kharif Seasons Type of soil in 2022	7196
7196	0	object	Kharif Seasons Type of water bodies in hectares 2022	7196
7196	0	object	Kharif Seasons Agro Ecological Sub Zone in 2022	7196
7196	0	float64	Kharif Seasons Seasonal average groundwater thickness (cm) in 2022	7196
7196	0	float64	Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2022	7196
7196	0	float64	Rabi Seasons Season Irrigated area in 2022	7196
7196	0	float64	Rabi Seasons Cropping density in 2022	7196

			Rabi Seasons Agricultural performance in 2022	7196
7196	0	float64		
			Rabi Seasons Agricultural Score in 2022	7196
7196	0	float64		
			Rabi Seasons Type of soil in 2022	7196
7196	0	object		
			Rabi Seasons Type of water bodies in hectares 2022	7196
7196	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2022	7196
7196	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2022	7196
7196	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2022	7196
7196	0	float64		
			Rabi Seasons Kharif Season Irrigated area in 2021	7196
7196	0	float64		
			Rabi Seasons Cropping density in 2021	7196
7196	0	float64		
			Rabi Seasons Agricultural performance in 2021	7196
7196	0	float64		
			Rabi Seasons Agricultural Score in 2021	7196
7196	0	float64		
			Rabi Seasons Type of soil in 2021	7196
7196	0	object		
			Rabi Seasons Type of water bodies in hectares 2021	7196
7196	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2021	7196
7196	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2021	7196
7196	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2021	7196
7196	0	float64		
			Kharif Seasons Kharif Season Irrigated area in 2021	7196
7196	0	float64		
			Kharif Seasons Cropping density in 2021	7196
7196	0	float64		
			Kharif Seasons Agricultural performance in 2021	7196
7196	0	float64		
			Kharif Seasons Agricultural Score in 2021	7196
7196	0	float64		
			Kharif Seasons Type of soil in 2021	7196
7196	0	object		
			Kharif Seasons Type of water bodies in hectares 2021	7196
7196	0	object		
			Kharif Seasons Agro Ecological Sub Zone in 2021	7196
7196	0	object		
			Kharif Seasons Seasonal average groundwater thickness (cm) in 2021	7196
7196	0	float64		
			Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2021	7196
7196	0	float64		
			Kharif Seasons Kharif Season Irrigated area in 2020	7196
7196	0	float64		
			Kharif Seasons Cropping density in 2020	7196
7196	0	float64		
			Kharif Seasons Agricultural performance in 2020	7196
7196	0	float64		

			Kharif Seasons Agricultural Score in 2020	7196
7196	0	float64		
			Kharif Seasons Type of soil in 2020	7196
7196	0	object		
			Kharif Seasons Type of water bodies in hectares 2020	7196
7196	0	object		
			Kharif Seasons Agro Ecological Sub Zone in 2020	7196
7196	0	object		
			Kharif Seasons Seasonal average groundwater thickness (cm) in 2020	7196
7196	0	float64		
			Kharif Seasons Seasonal average groundwater replenishment rate (cm) in 2020	7196
7196	0	float64		
			Rabi Seasons Kharif Season Irrigated area in 2020	7196
7196	0	float64		
			Rabi Seasons Cropping density in 2020	7196
7196	0	float64		
			Rabi Seasons Agricultural performance in 2020	7196
7196	0	float64		
			Rabi Seasons Agricultural Score in 2020	7196
7196	0	float64		
			Rabi Seasons Type of soil in 2020	7196
7196	0	object		
			Rabi Seasons Type of water bodies in hectares 2020	7196
7196	0	object		
			Rabi Seasons Agro Ecological Sub Zone in 2020	7196
7196	0	object		
			Rabi Seasons Seasonal average groundwater thickness (cm) in 2020	7196
7196	0	float64		
			Rabi Seasons Seasonal average groundwater replenishment rate (cm) in 2020	7196
7196	0	float64		
			Night light index	7196
7196	0	float64		
			Village score based on socio-economic parameters (Non normalised)	7196
7196	0	float64		
			Village score based on socio-economic parameters (0 to 100)	7196
7196	0	float64		
			Village category based on socio-economic parameters (Good, Average, Poor)	7196
7196	0	object		
			Land Holding Index source (Total Agri Area/ no of people)	7196
7196	0	float64		
			Road density (Km/ SqKm)	7196
7196	0	float64		

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import FunctionTransformer
from sklearn.pipeline import Pipeline

def drop_unnecessary_columns(df):
    df = df.copy()
    cols_to_drop = ['Unnamed: 0',
                    'FarmerID',
                    ' Village category based on socio-economic parameters (Good, Average, Poor)',
                    ' Village score based on socio-economic parameters (Non normalised)',
                    'Rabi Seasons Agro Ecological Sub Zone in 2022',
                    'Rabi Seasons Agro Ecological Sub Zone in 2021',

```

```

        'Kharif Seasons Agro Ecological Sub Zone in 2021',
        'Rabi Seasons Agro Ecological Sub Zone in 2020',
        'Kharif Seasons Agro Ecological Sub Zone in 2020',
        'Rabi Seasons Type of soil in 2022',
        'Rabi Seasons Type of soil in 2021',
        'Kharif Seasons Type of soil in 2021',
        'Rabi Seasons Type of soil in 2020',
        'Kharif Seasons Type of soil in 2020',
        'Rabi Seasons Type of water bodies in hectares 2021',
        'Rabi Seasons Type of water bodies in hectares 2020',
        'Kharif Seasons Type of water bodies in hectares 2021',
        'Kharif Seasons Type of water bodies in hectares 2022'
    ]

    return df.drop(columns=cols_to_drop, errors='ignore')

```

```

def handle_heavy_missing(df):
    df = df.copy()

    # Bureau data - fill with 0 and flag the missing status
    if 'Avg_Disbursement_Amount_Bureau' in df.columns:
        df['Missing_Bureau_Data'] = df['Avg_Disbursement_Amount_Bureau'].isnull().astype(int)
        df['Avg_Disbursement_Amount_Bureau'] = df['Avg_Disbursement_Amount_Bureau'].fillna(0)

    categorical_missing = ['Location', 'Address type', 'Ownership']
    existing_cols = [col for col in categorical_missing if col in df.columns]
    df[existing_cols] = df[existing_cols].fillna('Unknown')

    return df

```

```

def handle_light_missing(df):
    df = df.copy()
    light_missing_cols = [
        'Total_Land_For_Agriculture', 'Perc_of_house_with_6plus_room',
        'Women_15_19_Mothers_or_Pregnant_at_time_of_survey', 'perc_of_pop_living_in_hh_electri
        'perc_Households_with_Pucca_House_That_Has_More_Than_3_Rooms', 'mat_roof_Metal_GI_Asbe
        'perc_of_Wall_material_with_Burnt_brick', 'Households_with_improved_Sanitation_Facilit
        'perc_Households_do_not_have_KCC_With_The_Credit_Limit_Of_50k'
    ]

    existing_cols = [col for col in light_missing_cols if col in df.columns]
    for col in existing_cols:
        df[col] = df[col].fillna(df[col].median())

    return df

```

```

def extract_temperatures(df):
    df = df.copy()
    temp_columns = [
        'K022-Ambient temperature (min & max)', 'R022-Ambient temperature (min & max)',
        'K021-Ambient temperature (min & max)', 'R021-Ambient temperature (min & max)',
        'R020-Ambient temperature (min & max)'
    ]

    existing_cols = [col for col in temp_columns if col in df.columns]
    for col in existing_cols:

```

```

        # Convert to string and extract digits
        temps = df[col].astype(str).str.extract(r'(\d+\.\d*)\s*[-/,]\s*(\d+\.\d*)')
        df[f'{col}_min'] = temps[0].astype(float)
        df[f'{col}_max'] = temps[1].astype(float)
        df = df.drop(columns=[col])

    return df

```

```

def encode_ordinal_categories(df):
    """Maps ordinal text categories to ranked integers."""
    df = df.copy()

    # Define the logical mathematical order
    ordinal_mapping = {'Poor': 0, 'Average': 1, 'Good': 2}

    # The specific columns from your dataset that use this ranking
    ordinal_cols = [
        'K022-Village category based on Agri parameters (Good, Average, Poor)',
        'K022-Village category based on socio-economic parameters (Good, Average, Poor)',
        'R022-Village category based on Agri parameters (Good, Average, Poor)',
    ]

    # Apply the mapping only to columns that currently exist in the dataframe
    existing_cols = [col for col in ordinal_cols if col in df.columns]
    for col in existing_cols:
        # We use .map() to replace the strings with our integer dictionary
        df[col] = df[col].map(ordinal_mapping)

        # Optional: If there are missing values like "Unknown" or NaNs,
        # map() will turn them into NaNs. We can fill them with a neutral -1.
        df[col] = df[col].fillna(-1)

    return df

```

```

def encode_low_cardinality(df):
    """Applies One-Hot Encoding to categories with few unique values."""
    df = df.copy()

    low_card_cols = ['SEX', 'MARITAL_STATUS', 'REGION']
    existing_cols = [col for col in low_card_cols if col in df.columns]

    # get_dummies converts text to binary 0/1 columns
    df = pd.get_dummies(df, columns=existing_cols, drop_first=True)

    return df

```

```

def encode_high_cardinality(df):
    """Applies Frequency Encoding to geographical columns with many unique values."""
    df = df.copy()

    high_card_cols = ['State', 'CITY', 'DISTRICT', 'VILLAGE', 'K022-Nearest Mandi Name']
    existing_cols = [col for col in high_card_cols if col in df.columns]

    for col in existing_cols:
        # Calculate the frequency (percentage) of each category

```

```

freq_encoding = df[col].value_counts(normalize=True)

# Map the frequencies back to the dataframe
df[f'{col}_freq_encoded'] = df[col].map(freq_encoding)

# Drop the original text column
df = df.drop(columns=[col])

return df

```

```

from sklearn.base import BaseEstimator, TransformerMixin
import pandas as pd

class SafeOneHotEncoder(BaseEstimator, TransformerMixin):
    """
    A safe One-Hot Encoder that remembers training columns and
    prevents missing-column errors during Cross Validation.
    """
    def __init__(self, columns_to_encode):
        self.columns_to_encode = columns_to_encode
        self.train_columns_ = None # This will store the "memory" of our columns

    def fit(self, X, y=None):
        # 1. Create dummies on the training data
        X_encoded = pd.get_dummies(X, columns=self.columns_to_encode)

        # 2. Memorize the exact columns created during training
        self.train_columns_ = X_encoded.columns
        return self

    def transform(self, X):
        # 1. Create dummies on the new/validation data
        X_encoded = pd.get_dummies(X, columns=self.columns_to_encode)

        # 2. Force the new data to have the exact same columns as the training data
        # If a category was missing in the new data, it fills the column with 0
        # If a NEW category appeared in the test data, it safely ignores it
        X_encoded = X_encoded.reindex(columns=self.train_columns_, fill_value=0)

        return X_encoded

```

```

from sklearn.preprocessing import OrdinalEncoder

def preprocess_for_trees(df: pd.DataFrame) -> pd.DataFrame:
    """
    Preprocesses a DataFrame containing Zipcode, Location, Address type,
    and Ownership for tree-based machine learning models.
    """
    # Create a copy to avoid modifying the original dataframe
    df_clean = df.copy()

    # 1. Temporarily replace 'Unknown' with NaN to cleanly split coordinates
    df_clean.replace('Unknown', np.nan, inplace=True)

    # 2. Split 'Location' into 'Latitude' and 'Longitude'

```

```

if 'Location' in df_clean.columns:
    df_clean[['Latitude', 'Longitude']] = df_clean['Location'].str.split(',', expand=True)
    df_clean.drop('Location', axis=1, inplace=True)

# 3. Handle Numeric Missing Values (Imputing with -999)
if 'Latitude' in df_clean.columns and 'Longitude' in df_clean.columns:
    df_clean['Latitude'] = df_clean['Latitude'].fillna(-999)
    df_clean['Longitude'] = df_clean['Longitude'].fillna(-999)

# 4. Handle Categorical Missing Values
cat_cols = ['Zipcode', 'Address type', 'Ownership']

# Ensure we only process columns that actually exist in the dataframe
cat_cols = [col for col in cat_cols if col in df_clean.columns]

if cat_cols:
    # Fill NaNs back to 'Unknown' so the model treats it as a specific category
    df_clean[cat_cols] = df_clean[cat_cols].fillna('Unknown')

    # Force Zipcode to be a string before encoding
    if 'Zipcode' in df_clean.columns:
        df_clean['Zipcode'] = df_clean['Zipcode'].astype(str)

    # 5. Apply Ordinal Encoding
    encoder = OrdinalEncoder()
    df_clean[cat_cols] = encoder.fit_transform(df_clean[cat_cols])

return df_clean

# --- How to use it ---
# original_df = pd.read_csv('your_data.csv')
# processed_df = preprocess_for_trees(original_df)
# print(processed_df.head())

```

```

import pandas as pd
import ast
from sklearn.preprocessing import MultiLabelBinarizer

def automate_water_encoding(df):
    """
    Parses, splits comma-separated strings, and multi-hot encodes
    the water body columns for tree models.
    """
    df_clean = df.copy()

    target_columns = [
        'Rabi Seasons Type of water bodies in hectares 2022',
        'Kharif Seasons Type of water bodies in hectares 2020'
    ]

    water_columns = [col for col in target_columns if col in df_clean.columns]

    if not water_columns:
        print("Columns already processed or not found. Returning original dataframe.")
        return df_clean

```

```

# --- THE FIX: An aggressively cleaning parser ---
def parse_and_split(val):
    if pd.isna(val) or val in ['[None]', 'None', '[]']:
        return []

    # 1. Evaluate string list representations
    if isinstance(val, str):
        try:
            val = ast.literal_eval(val)
        except (ValueError, SyntaxError):
            val = [val] # Wrap in list if it's just a raw string

    if not isinstance(val, list):
        val = [val]

    # 2. Split by comma and clean whitespace
    final_categories = set() # Using a set prevents duplicates
    for item in val:
        if item is None or str(item).strip().lower() == 'none':
            continue

        # This splits 'riverbank, water' into ['riverbank', ' water']
        for sub_item in str(item).split(','):
            clean_word = sub_item.strip() # Removes trailing/leading spaces
            if clean_word:
                final_categories.add(clean_word)

    return list(final_categories)

# Apply the aggressive parser
for col in water_columns:
    df_clean[col] = df_clean[col].apply(parse_and_split)

# The rest remains exactly the same
all_water_lists = df_clean[water_columns].sum(axis=1)

mlb = MultiLabelBinarizer()
mlb.fit(all_water_lists)

for col in water_columns:
    encoded_matrix = mlb.transform(df_clean[col])

    prefix = "Kharif" if "Kharif" in col else "Rabi"
    # We replace spaces with underscores just to make column names look cleaner
    encoded_col_names = [f"{prefix}_has_{category.replace(' ', '_')}" for category in mlb.classes_]

    encoded_df = pd.DataFrame(encoded_matrix, columns=encoded_col_names, index=df_clean.index)

    df_clean = pd.concat([df_clean, encoded_df], axis=1)
    df_clean.drop(columns=[col], inplace=True)

return df_clean

```

```

import pandas as pd
import numpy as np

```

```

def extract_text_features(df):
    """
    Extracts key descriptive words from Soil and Agro Zone columns
    and converts them into binary (1/0) features.
    """
    df_clean = df.copy()

    # 1. Define the exact column names from your data
    soil_col = 'Kharif Seasons Type of soil in 2022'
    zone_col = 'Kharif Seasons Agro Ecological Sub Zone in 2022'

    # Safety check: ensure columns exist
    if soil_col not in df_clean.columns or zone_col not in df_clean.columns:
        print("Columns not found. Please check spelling.")
        return df_clean

    # Fill NaNs with empty strings so the text search doesn't crash
    df_clean[soil_col] = df_clean[soil_col].fillna('')
    df_clean[zone_col] = df_clean[zone_col].fillna('')

    # 2. Define the Keywords to search for in SOIL
    # Format: {'New_Column_Name': 'search_term'}
    soil_keywords = {
        'Soil_is_Black': 'black',
        'Soil_is_Red': 'red',
        'Soil_is_Loamy': 'loamy|loam', # The | acts as an OR
        'Soil_is_Mixed': 'mixed',
        'Soil_is_Deep': 'deep',
        'Soil_is_Shallow': 'shallow',
        'Soil_is_Medium': 'medium'
    }

    # 3. Define the Keywords to search for in AGRO ZONE
    zone_keywords = {
        'Zone_is_Hot': 'hot',
        'Zone_is_Subhumid': 'subhumid|sub humid',
        'Zone_is_SemiArid': 'semi-arid|semi arid',
        'Zone_is_Central_Highlands': 'central highlands',
        'Zone_is_Deccan': 'deccan',
        'Zone_is_Eastern_Ghats': 'eastern ghats',
        'Zone_is_Malwa': 'malwa'
    }

    # 4. Perform the extraction for Soil
    for new_col, keyword in soil_keywords.items():
        # str.contains searches for the keyword. case=False ignores capitalization.
        # astype(int) converts True/False into 1/0
        df_clean[new_col] = df_clean[soil_col].str.contains(keyword, case=False, regex=True).astype(int)

    # 5. Perform the extraction for Agro Zone
    for new_col, keyword in zone_keywords.items():
        df_clean[new_col] = df_clean[zone_col].str.contains(keyword, case=False, regex=True).astype(int)

    # 6. Drop the original messy text columns
    df_clean.drop(columns=[soil_col, zone_col], inplace=True)

```

```
return df_clean
```

```
# --- How to run it ---  
# df = extract_text_features(df)
```

```
# Load your raw data  
df_raw = pd.read_csv('PS/train.csv')  
  
# Run the pipeline  
df_cleaned = (df_raw  
              .pipe(drop_unnecessary_columns)  
              .pipe(handle_heavy_missing)  
              .pipe(handle_light_missing)  
              .pipe(extract_temperatures)  
              .pipe(encode_ordinal_categories)  
              .pipe(encode_low_cardinality)  
              .pipe(encode_high_cardinality)  
              .pipe(preprocess_for_trees)  
              .pipe(automate_water_encoding)  
              .pipe(extract_text_features)  
              )
```

```
df_cleaned.to_csv('train_cleaned.csv', index=False)
```

```
step1 = FunctionTransformer(drop_unnecessary_columns)  
step2 = FunctionTransformer(handle_heavy_missing)  
step3 = FunctionTransformer(handle_light_missing)  
step4 = FunctionTransformer(extract_temperatures)  
step5 = FunctionTransformer(encode_ordinal_categories)  
step6 = FunctionTransformer(encode_low_cardinality)  
step7 = FunctionTransformer(encode_high_cardinality)  
step8 = FunctionTransformer(preprocess_for_trees)  
step9 = FunctionTransformer(automate_water_encoding)  
step10 = FunctionTransformer(extract_text_features)  
cols_to_ohe = ['MARITAL_STATUS', 'REGION', 'SEX']  
step_ohe = SafeOneHotEncoder(columns_to_encode=cols_to_ohe)
```

```
# Build the scikit-learn pipeline  
preprocessing_pipeline = Pipeline(steps=[  
    ('drop_cols', step1),  
    ('heavy_missing', step2),  
    ('light_missing', step3),  
    ('extract_temps', step4),  
    ('encode_ordinal_categories', step5),  
    ('safe_one_hot_encoding', step_ohe),  
    ('encode_low_cardinality', step6),  
    ('encode_high_cardinality', step7),  
    ('preprocess_for_trees', step8),  
    ('drop_unnecessary_columns', step9),  
    ('handle_heavy_missing', step10)  
])
```

```
# Execute it  
df_cleaned = preprocessing_pipeline.fit_transform(df_raw)  
df_cleaned.to_csv('train_cleaned.csv', index=False)
```

```

import numpy as np
import xgboost as xgb
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.compose import TransformedTargetRegressor
from sklearn.metrics import mean_absolute_percentage_error, make_scorer

# ... [Your step1 through step10 definitions stay exactly the same] ...

# 1. Define the base XGBoost model
base_xgb = xgb.XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42,
    verbosity=0
)

# 2. Wrap the model to automatically log-transform the target during training,
# and exponentially reverse it during prediction.
# We use np.log1p (log(1+x)) instead of np.log to safely handle zeros in your target data.
xgb_log_model = TransformedTargetRegressor(
    regressor=base_xgb,
    func=np.log1p,
    inverse_func=np.expm1,
)

# 3. Build the full pipeline
full_pipeline = Pipeline(steps=[
    ('drop_cols', step1),
    ('heavy_missing', step2),
    ('light_missing', step3),
    ('extract_temps', step4),
    ('encode_ordinal_categories', step5),
    ('safe_one_hot_encoding', step_ohe),
    ('encode_low_cardinality', step6),
    ('encode_high_cardinality', step7),
    ('preprocess_for_trees', step8),
    ('automate_water_encoding', step9),
    ('extract_text_features', step10),
    ('model', xgb_log_model) # The wrapped model goes here
])

# 4. Prepare data and split 80/20
target_column = 'Target_Variable/Total Income'
X = df_raw.drop(columns=[target_column])
y = df_raw[target_column]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Setup 5-Fold Cross Validation on the 80% Training Data
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mape_scorer = make_scorer(mean_absolute_percentage_error)

print("Starting 5-Fold Cross Validation...")

```

```

# cross_val_score automatically handles isolating the folds, running the full pipeline
# (preventing data leakage), and calculating the MAPE for each fold.
# Added verbose=3 to print progress during the 5 folds
cv_scores = cross_val_score(
    full_pipeline,
    X_train,
    y_train,
    cv=kf,
    scoring=mape_scorer,
    verbose=3
)

print(f"CV MAPE Scores for each fold: {cv_scores}")
print(f"Average CV MAPE: {cv_scores.mean():.4f}")

# 6. Final Model Training and Holdout Evaluation
print("\nTraining final pipeline on full 80% train set...")
full_pipeline.fit(X_train, y_train)

print("Predicting on 20% unseen test set...")
final_preds = full_pipeline.predict(X_test)
final_mape = mean_absolute_percentage_error(y_test, final_preds)

print(f"Final Holdout MAPE: {final_mape:.4f}")

```

Starting 5-Fold Cross Validation...

```

[CV] END ..... score: (test=0.243) total time= 2.5s
[CV] END ..... score: (test=0.230) total time= 2.8s
[CV] END ..... score: (test=0.231) total time= 2.6s
[CV] END ..... score: (test=0.252) total time= 2.7s
[CV] END ..... score: (test=0.245) total time= 2.8s
CV MAPE Scores for each fold: [0.24252979 0.2299075 0.23141694 0.25210327 0.24538423]
Average CV MAPE: 0.2403

```

Training final pipeline on full 80% train set...

[Parallel(n_jobs=1)]: Done 5 out of 5 | elapsed: 13.7s finished

Predicting on 20% unseen test set...

Final Holdout MAPE: 0.2321